

Documentación Técnica

Sistema de Reservas

Ali, Lucas Exequiel

04/05/2025

1. Introducción

Este documento presenta una visión general del sistema de reservas para el **Teatro Gran Espectáculo**. La aplicación ha sido desarrollada en **Java**, utilizando **Spring Boot** como framework principal, y gestionada dentro del entorno de desarrollo **Eclipse**. A lo largo del documento, exploraremos más en profundidad como puede ser, las funcionalidades importantes y la lógica de negocio que permite una gestión eficiente de las reservas. Como base de datos utilice dos opciones, H2 y PostgreSQL, más adelante podrás elegir cual puedes utilizar.

2. Requisitos Técnicos

Backend (Java + Spring Boot)

Para levantar y ejecutar el backend del sistema de reservas se requieren los siguientes elementos:

1. **Java 17**
Lenguaje de programación utilizado, compatible con las versiones modernas de Spring Boot.
2. **Spring Boot**
Framework utilizado para construir la aplicación de backend, facilitando la configuración y el desarrollo de servicios REST.
3. **PostgreSQL(opcional)**
Sistema de gestión de bases de datos utilizado para persistir la información del sistema. Se configura como base de datos principal en el entorno de producción. (Se detalla su configuración en otra sección).
4. **Maven**
Herramienta de gestión de dependencias y construcción del proyecto.
5. **IDE compatible con Java**
Como Eclipse, IntelliJ IDEA u otro entorno de desarrollo que permita compilar y ejecutar aplicaciones Spring Boot.
6. **Herramientas adicionales (opcional)**
 - Postman o Swagger para probar y documentar las APIs REST.
 - H2 (base de datos en memoria) puede utilizarse durante el desarrollo o pruebas unitarias.

Frontend (React + Vite)

Para ejecutar correctamente el frontend de la aplicación, se requiere lo siguiente:

1. **Node.js (versión 18 o superior)**
Motor de JavaScript necesario para ejecutar Vite y administrar dependencias.
2. **Vite**
Herramienta de construcción moderna utilizada para desarrollar el frontend de manera rápida y eficiente.
3. **React**
Librería de JavaScript para construir interfaces de usuario basadas en componentes.
4. **Gestión de paquetes (npm)**
Para instalar dependencias del proyecto.
5. **Editor de código**
Como Visual Studio Code, recomendado para trabajar con React y herramientas modernas de frontend.

3. Arquitectura del Sistema

La arquitectura del sistema sigue el patrón de **arquitectura en capas**, lo que permite una separación clara de responsabilidades, facilita el mantenimiento y mejora la escalabilidad del proyecto.

Backend

El backend está construido con **Java y Spring Boot**, y se estructura en los siguientes paquetes:

- **Controllers:** Contienen los endpoints REST que reciben y responden a las solicitudes HTTP. Se encargan de delegar la lógica a las capas de servicio.
- **Services:** Implementan la lógica de negocio de la aplicación. En algunos casos, se estructuran mediante interfaces e implementaciones para fomentar la abstracción y la inyección de dependencias.
- **Entities:** Representan las clases del modelo de datos, mapeadas a las tablas de la base de datos mediante anotaciones JPA.
- **Repositories:** Interactúan con la base de datos, extendiendo `JpaRepository` para acceder y manipular las entidades persistentes.

Además, se utiliza el patrón de **eventos de dominio** (Domain Events) para ejecutar acciones automáticas al producirse ciertos eventos.

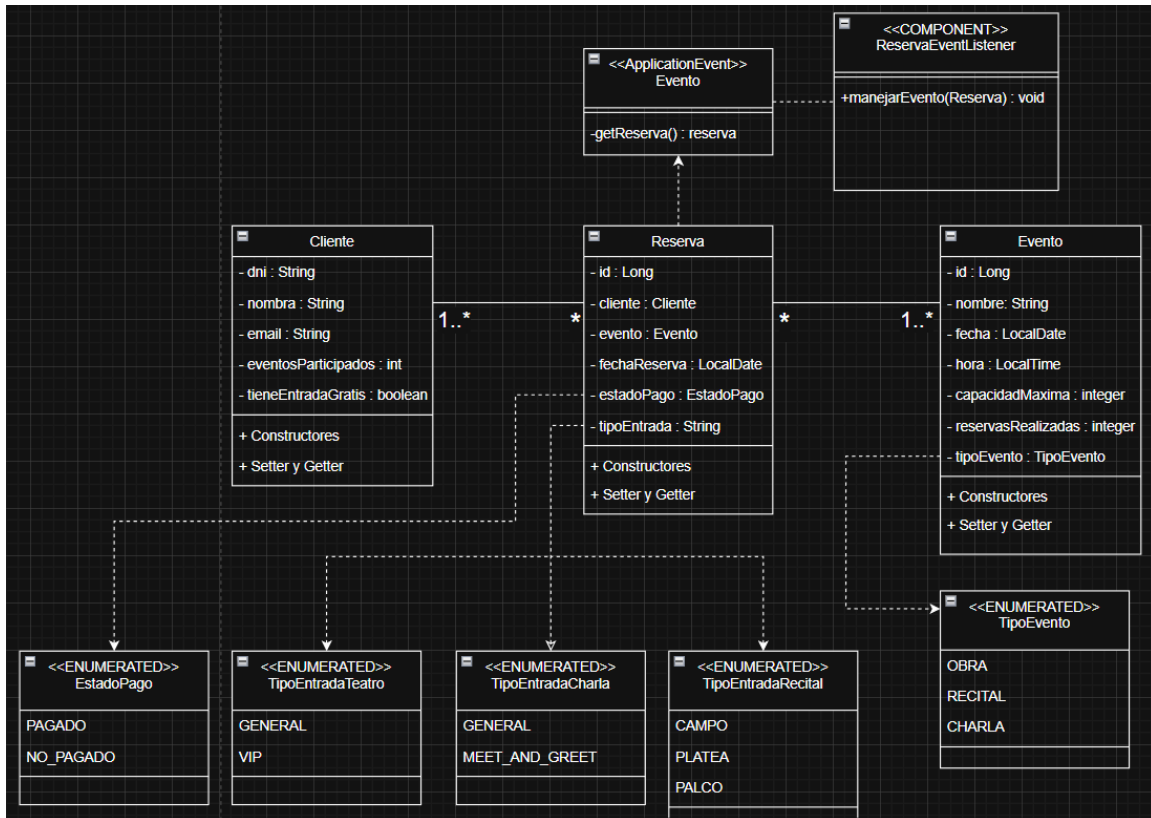
Frontend

El frontend fue desarrollado con **React** y gestionado con **Vite**, respetando una estructura modular y clara:

- **Componentes:** Elementos como NavBar para utilizarlo en todas las paginas.
- **Páginas:** Vistas principales del sistema (inicio, cliente, evento, reservas).
- **Servicios:** Encapsulan las llamadas a la API REST del backend, centralizando la lógica de comunicación.

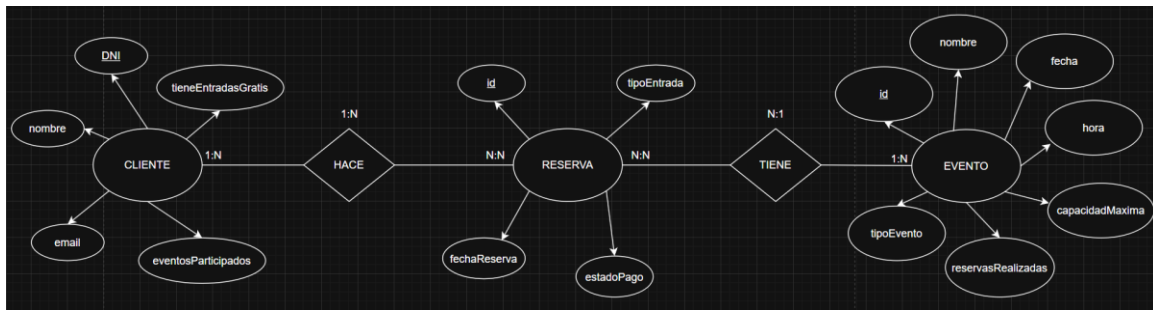
4. Diagrama de Clase

El diagrama de clases representa las entidades y relaciones clave dentro del sistema de reservas. Se incluyen las entidades Cliente, Evento, Reserva y sus respectivas relaciones.



5. Diagrama Entidad-Relación (ER)

El diagrama entidad-relación describe las tablas y relaciones en la base de datos. El sistema de reservas maneja principalmente las siguientes entidades:



- **Cliente → Reserva (1 : N):** Un **cliente** puede realizar múltiples reservas, pero cada reserva pertenece a un único cliente.
- **Reserva → Evento (N : 1):** Una **reserva** corresponde a un único evento, pero un evento puede tener varias reservas.

6. Documentación con Postman

La documentación si deseas utilizar Postman:

1. Clientes

- **Listar Clientes** (GET localhost:8080/clientes): Recupera la lista de todos los clientes registrados.
- **Crear Cliente** (POST localhost:8080/clientes): Agrega un nuevo cliente al sistema.
- **Editar Cliente** (PUT localhost:8080/clientes/{dni}): Actualiza la información de un cliente existente.
- **Eliminar Cliente** (DELETE localhost:8080/clientes/{dni}): Elimina un cliente del sistema.
- **Reservas por Cliente** (GET localhost:8080/clientes/{dni}/reservas): Obtiene todas las reservas asociadas a un cliente específico.

2. Eventos

- **Listar Eventos** (GET localhost:8080/eventos): Recupera la lista de todos los eventos disponibles.
- **Crear Evento** (POST localhost:8080/eventos): Agrega un nuevo evento al sistema.
- **Editar Evento** (PUT localhost:8080/eventos/{id}): Actualiza la información de un evento existente.
- **Eliminar Evento** (DELETE localhost:8080/eventos/{id}): Elimina un evento del sistema.

3. Reservas

- **Listar Reservas** (GET `localhost:8080/reservas`): Recupera la lista de todas las reservas realizadas.
- **Reservar Evento** (POST `localhost:8080/reservas`): Crea una nueva reserva para un evento.
- **Editar Reserva** (PUT `localhost:8080/reservas/{id}`): Actualiza la información de una reserva existente.
- **Eliminar Reserva** (DELETE `localhost:8080/reservas/{id}`): Elimina una reserva del sistema.

Dejo el espacio de trabajo para postman, pero lo aclare anteriormente por si no le funciona o no quiere descargar postman app.

<https://www.postman.com/lunar-module-participant-20021517/sistema-reservas/collection/sktxznc/teatro?action=share&creator=30348671>

7. Requisitos Cumplidos

El sistema de reservas del **Teatro Gran Espectáculo** cumple con los siguientes requisitos, asegurando una gestión eficiente de eventos y una experiencia fluida para los clientes.

Tipos de Eventos y Entradas

Los eventos disponibles en el sistema se clasifican en diferentes categorías, cada una con sus propios tipos de entradas, representados mediante **enumeraciones (enum)** en el código.

Obras de teatro → Entrada **GENERAL** o **VIP**

Recitales → Entrada **CAMPO**, **PLATEA** o **PALCO**

Charlas y conferencias → Entrada **GENERAL** o **MEET_AND_GREET**

Este enfoque con enum permite un manejo estructurado de los tipos de entrada y evita errores en la selección de valores.

Registro de reservas

Cada reserva generada en el sistema incluye información clave para su correcta gestión:

Fecha y hora del evento → Obtenida desde la entidad Evento.

Nombre del asistente → Recuperado desde Cliente.

Tipo de entrada → Depende del tipo de evento (`tipoEntrada`).

Además, se utiliza `EventListener` dentro del **Event Domain**, lo que permite disparar eventos internos cuando se crea una reserva. Esto facilita las acciones como la notificación automática.

Fidelización: entrada gratuita

Luego de asistir a 5 eventos diferentes, en un año, el sistema marca **tieneEntradaGratis = true** en el cliente.

Usando el **EventListener**, del antes mencionado “Event Domain”, se encarga de observar las reservas realizadas, incluye, la finalidad que puede observar si el cliente llevo al 5 evento diferente, y mandarte un email (simulado con logger) informándote la buena noticia.

Disponibilidad actualizada

Antes de crear una reserva, se verifica si hay cupos.

En la primera página del frontend, se puede observar los eventos disponibles ordenados por fecha, y mostrándote información de cuantos lugares disponibles le quedan.

Si se supera la capacidad, se lanza una excepción y no se guarda la reserva.

Los eventos tienen `reservasRealizadas` y `capacidadMaxima`.

Base de datos: “No tenemos una base de datos”

Ahora utilizamos como base de datos **H2**, es recomendado utilizarla cuando estamos verificando que ande correctamente, una vez todo se verifique que funciona correctamente, están testeados todos los casos, tenemos una buena documentación y podemos empezar a vender el sistema, tenemos que realizar una base de datos utilizando **PostgreSQL**.

Con una buena configuración, podemos realizar la dependencia de cada entidad, por ejemplo si se elimina un cliente, se deberá eliminar sus reservas no pagadas realizadas, dejamos las que fueron pagadas para un análisis de datos posteriormente.

Una de las ventajas que utilice fue `@Transactional`, en algunos services que implementen actuar en varios lugares, ya que, si falla a mitad de camino, no se guarde lo que se vino realizando, si elimino al cliente y a mitad de eliminar la reserva sucede algo y falla, no se concluirá la eliminación al cliente y se volver como si no hubiera pasado.

8. Base de Datos

En el backend del sistema de reservas, la configuración de la base de datos se define en el archivo `application.properties`, ubicado en la ruta: `src/main/resources/application.properties`. Aquí se especifican los parámetros de conexión y las reglas de creación de tablas.

1. Opción H2 (Base de Datos Embebida)

Se recomienda comenzar con esta opción si lo necesario es probar el sistema.

```
-----  
spring.datasource.url=jdbc:h2:mem:testdb  
  
spring.datasource.driver-class-name=org.h2.Driver  
  
spring.datasource.username=admin  
  
spring.datasource.password=1234  
  
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect  
  
spring.h2.console.enabled=true  
  
spring.jpa.hibernate.ddl-auto=create-drop  
-----
```

Si necesita ver los datos guardados en la base de datos, deberá entrar a <http://localhost:8080/h2-console>, donde pedirá datos para ingresar:

Debe verificar que **JDBC URL**, **User Name** (admin) y **Password** (1234) tengan el mismo valor que colocamos en el código una vez que coincidan y colocamos **Connect**, abre la DB:

Podemos observar que se guarda en la base de datos.

2. Opción PostgreSQL

1. Descargar PostgreSQL (En el caso que no lo tenga y desee usarlo)
2. La cuenta que hay que crear puede ser la default como indica el archivo, username, postgres y password 1234, si crea otra cuenta, puede editar esos valores del archivo “application.properties”.
3. Crear una base de datos vacía llamada “teatro”

```
spring.datasource.url=jdbc:postgresql://localhost:5432/teatro
spring.datasource.username=postgres
spring.datasource.password=1234
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

9. Test

Como pruebas unitarias, realice dos, una para tener en cuenta que no es posible crear una reserva cuando ya no hay lugar en el evento. Lo podemos encontrar en

"src/test/java/com/teatro/app/service/ReservaServiceTest"

```
@Test
@DisplayName("Debe lanzar error si no hay lugar")
void noDebePermitirReservaCuandoEventoEstaLleno() {
    Cliente cliente = new Cliente("123", "Pedro", "pedro@mail.com", 0, false);
    Evento evento = new Evento();
    evento.setId(1L);
    evento.setCapacidadMaxima(1);
    evento.setReservasRealizadas(1);

    when(clienteRepo.findById("123")).thenReturn(Optional.of(cliente));
    when(eventoRepo.findById(1L)).thenReturn(Optional.of(evento));

    RuntimeException ex = assertThrows(RuntimeException.class, () -> {
        ReservaDTO reserva = new ReservaDTO("123", 1L, "VIP", EstadoPago.PAGADO);
        reservaService.crearReserva(reserva);
    });

    assertEquals("Evento sin disponibilidad", ex.getMessage());
    System.out.println(ex.getMessage());
}
```

Y como segunda prueba unitaria verificar que es posible crear un cliente y luego buscarlo por su DNI. Lo podemos encontrar en

"src/test/java/com/teatro/app/service/ClienteServiceTest"

```

@Test
@DisplayName("Deberia dar correcto, si el cliente se guarda y se encuentra")
void guardarYBuscarCliente() {
    Cliente cliente = new Cliente("123", "Ana", "ana@email.com", 0, false);

    when(clienteRepo.save(any())).thenReturn(cliente);
    clienteService.crearCliente(cliente);

    when(clienteRepo.findById("123")).thenReturn(Optional.of(cliente));
    Optional<Cliente> resultado = clienteService.buscarPorDni("123");
    assertTrue(resultado.isPresent());
    assertEquals("Ana", resultado.get().getNombre());
}

```

10. Mejoras a futuro

Cuentas para usuarios y administradores

Actualmente, cualquier persona que accede al frontend tiene la capacidad de crear, editar o eliminar clientes, eventos y reservas, así como también ver las reservas de otros usuarios. Una mejora importante y próxima, sería la implementación de autenticación y autorización utilizando **JWT** y **OAuth2**, con el objetivo de gestionar accesos según roles de usuario.

La propuesta consiste en definir al menos dos tipos de roles:

- **Administrador:** con permisos totales para crear, editar y eliminar entidades, así como ver toda la información del sistema.
- **Usuario:** podrá ver y editar únicamente sus propias reservas. No tendrá permisos para modificar el estado de pago de reservas ni acceder a datos de otros usuarios.

Esta mejora reforzará la seguridad del sistema y garantizará un uso controlado y segmentado de la plataforma.

Domain Events

Se implementó un mecanismo de **eventos de dominio**, que permite ejecutar acciones automáticas a partir de ciertas condiciones, en este caso cuando es creada una reserva.

Por ejemplo, al crear una reserva:

- Se verifica si el cliente ha asistido a 5 eventos diferentes. Si es así, se le comunica por mail y se le otorga una entrada gratuita para el siguiente evento.
- En este caso, a mejorar, únicamente si una vez creada la reserva, faltan dos días para el evento y la reserva no está pagada, se dispara un evento para enviar un correo recordatorio al cliente.
- Cuando es eliminado un evento, se encarga de enviar un correo al usuario que hizo la reserva.

A futuro, este sistema puede mejorarse mediante un **proceso de larga duración (long-running process)** que se ejecute diariamente (por ejemplo, a las 8:00 a.m.), revisando todas las reservas pendientes de pago cuyos eventos estén próximos. Si se detecta alguna, se enviará automáticamente una notificación por correo electrónico para informar al cliente que debe completar el pago.

Filtros y Búsquedas Avanzadas

Se propone implementar funcionalidades de búsqueda y filtrado tanto en la interfaz de usuario como en el backend para mejorar la experiencia de navegación y administración de datos.

- Permitir a los usuarios buscar eventos por **nombre, fecha o tipo de evento**, facilitando así la localización de espectáculos específicos.
- Incorporar filtros para las reservas, permitiendo separarlas por **estado de pago** (pagado, no pagado) y por **tipo de entrada**.
- Estas búsquedas deberían ser eficientes, utilizando consultas optimizadas en el backend.

Ampliación de Pruebas Unitarias y de Integración

Actualmente se cuenta con dos pruebas unitarias, pero próximamente, se recomienda ampliar las pruebas unitarias con la finalidad de garantizar la estabilidad y calidad del código.

- Incrementar la cantidad y profundidad de **pruebas unitarias**, especialmente para los servicios de negocio (`Service`) y controladores (`Controller`).
- Implementar **pruebas de integración** que simulen flujos completos de uso (por ejemplo, crear cliente → hacer reserva → cancelar evento).

Ampliación de Pruebas Unitarias y de Integración

Actualmente solo manejo las excepciones cuando un cliente no es encontrado, lo cual es insuficiente en varios entornos, por lo que se debería hacer un manejo de excepciones.

- Creando la clase **@ControllerAdvice** donde vamos a capturar todas las excepciones que se lanzen en los controladores y servicios, tanto las comunes como por ejemplo **IllegalArgumentException** y las personalizadas como la que tenemos **ClienteNoEncontradoException**.
- Mejora la facilidad de detectar errores.

10. Ejecutar el sistema

1. Verificar que cumple los **Requisitos Técnicos**.

1. Si cumple los requisitos, entonces el primer paso será obtener el sistema, las dos carpetas:
 - a. Una forma es poder clonar el repositorio:
git clone https://github.com/LucasAli18/Teatro_2025.git
 - b. La segunda forma es descargar desde la página de github el ZIP. Por las dudas de igual manera enviare el sistema descargado.
2. Una vez obtenemos la carpeta del repositorio, dentro vamos a tener la carpeta del backend y la del frontend
3. La carpeta del backend debe ser importada a la IDE que usted tenga (Eclipse, IntelliJ).
 - a. Si utiliza H2 como esta propuesto al principio y como viene por defecto el backend, no necesita realizar más que ejecutar el sistema.
 - b. Si desea utilizar PostgreSQL, primero deberá seguir los pasos explicados anteriormente para poder continuar ejecutando el sistema.
4. Una vez elegida la base de datos, deberá darle para ejecutar el backend.
5. Una vez que esté funcionando el backend, deberá utilizar la carpeta de Teatro-Frontend y ejecutarla con visual studio code.
6. Una vez en visual studio code, abrir terminal colocar primero **"npm install"** y luego **"npm run dev"** para que empiece a ejecutarse (claramente cumpliendo los requisitos mencionados al principio).
7. Se debería estar ejecutando en <http://localhost:5173/>, ingresa y debería aparecerle el menú.



[Eventos](#) | [Clientes](#) | [Eventos](#) | [Reservas](#)

Eventos

8. Recomiendo instalar **PostmanApp** para poder utilizar la documentación de postman que deje anteriormente. Igual la dejamos acá:
<https://www.postman.com/lunar-module-participant-20021517/sistema-reservas/collection/sktxznc/teatro?action=share&creator=30348671>
9. Hay varias formas para probar cada el sistema. Podes crear todo lo necesario por postman, o llegar a un cierto punto en postman y seguir desde el frontend o crear todo desde el frontend. Paso fotos del como debería verse el FrontEnd:

Clientes

[Eventos](#) | [Clientes](#) | [Eventos](#) | [Reservas](#)

Clientes

DNI	Nombre	Email	Crear
----------------------------------	-------------------------------------	------------------------------------	--------------------------------------

Eventos

[Eventos](#) | [Clientes](#) | [Eventos](#) | [Reservas](#)

Eventos

Crear

Reservas

[Eventos](#) | [Clientes](#) | [Eventos](#) | [Reservas](#)

Crear Reserva

Reservar

Reservas existentes